

EMG-to-QWERTY Final Project Report Preview

Abstract

We study single-user EMG-to-QWERTY decoding on the emg2qwerty dataset and ask which modeling choices matter most in a small-data regime. Starting from the course TDS CNN baseline, we compare recurrent, hybrid, and attention-heavy architectures, then run ablations over temporal resolution, context design, recurrent architecture, and data availability. The main empirical pattern is that efficient recurrent models outperform both the baseline CNN and larger attention-heavy alternatives, while lower effective frame rate and longer temporal context improve generalization. On the best spectrogram-based encoder, beam search with a 6-gram character language model reduces test CER from 14.11 to 8.69. Guided by the hop-length ablation, we then replace the handcrafted spectrogram with a learned-downsampling Raw-CNN+BiGRU: a three-layer strided CNN whose total stride matches the previously optimal 48x temporal downsampling, followed by a two-layer BiGRU. This Raw-CNN+BiGRU becomes the strongest greedy encoder we observed, reaching 9.933 validation CER and 11.476 test CER, and its best decoded test result further improves to 6.938. Overall, the project suggests that in this single-user setting, compact recurrent sequence modeling and learned temporal compression matter more than simply increasing model size or attention capacity.

1 Introduction

Surface electromyography (sEMG) records muscle activity from the wrist and offers a promising path toward non-invasive typing interfaces. In the emg2qwerty task, the goal is to map multichannel EMG sequences to the characters typed on a QWERTY keyboard. Because the typed characters occur at irregular timestamps and are not pre-aligned with the input sequence, the task is naturally formulated as a sequence prediction problem with Connectionist Temporal Classification (CTC) loss and Character Error Rate (CER) evaluation.

The course starter repository provides a convolutional baseline based on a TDS encoder. However, EMG typing is fundamentally sequential, and the project guidelines explicitly encourage exploration beyond CNNs. This raises a practical question: in a single-user, limited-data regime, which modeling choices produce the largest CER improvement? Candidate answers include recurrent encoders, CNN-RNN hybrids, attention-heavy models, decoder-side language modeling, and preprocessing choices that change the effective temporal resolution seen by the model.

Our project focuses on the personalized single-user setting from the course split and organizes the investigation around three stages. First, we compare broad encoder families to determine whether recurrent models outperform the CNN baseline. Second, we run targeted ablations over sampling rate, context design, recurrent architecture, and data availability to identify the dominant design factors. Third, we separate decoder gain from encoder gain and then introduce a learned-downsampling Raw-CNN+BiGRU that replaces the handcrafted spectrogram with a three-layer CNN front end. The resulting storyline is coherent: architecture selection points toward recurrence, ablations identify the value of coarse temporal resolution and longer context, beam search reveals the size of decoder-side gain, and the final Raw-CNN+BiGRU turns the temporal-compression insight into the best overall system.

2 Methods

2.1 Experimental Setup

All experiments were conducted on the personalized single-user split for subject 89335547 provided by the course repository. The split contains 16 training sessions, 1 validation session, and 1 test session. We report

Character Error Rate (CER) on validation and test, with lower being better. Unless otherwise noted, models were trained with CTC loss and decoded with greedy CTC decoding; beam-search decoding was applied only in the decoder study and in the final post-hoc evaluation of the strongest checkpoints.

The input signal is raw sEMG sampled at 2 kHz from two wrist bands, with 16 channels per band. For the spectrogram-based model family, the training transform stack was `ToTensor` $\rightarrow$ `RandomBandRotation` $\rightarrow$ `TemporalAlignmentJitter` $\rightarrow$ `LogSpectrogram` $\rightarrow$ `SpecAugment`, while validation and test used `ToTensor` $\rightarrow$ `LogSpectrogram`. This kept the downstream loss and decoder interface fixed while isolating architecture and context-design effects. The learned-downsampling Raw-CNN+BiGRU is the main exception: it used only `ToTensor` and learned its temporal downsampling directly from raw EMG.

Window length, stride, and padding are reported in raw EMG samples. At 2 kHz, `hop=48` corresponds to about 41.7 spectrogram frames per second, `window_length=20000` corresponds to 10 seconds of signal, `stride=12000` corresponds to 6 seconds, and `padding=[900,200]` corresponds to about 450 ms of past context plus 100 ms of future context. One protocol detail is important for interpreting context experiments: train and validation use the configured windowing scheme, whereas test feeds each full session at once with no windowing or contextual padding. We used broad 40-epoch screening runs to search the space, then reran only promising candidates for longer 100-, 120-, or 150-epoch confirmation runs.

2.2 Model Families and Study Design

The reference system was the course baseline TDS CNN. We compared it against BiLSTM and BiGRU encoders, hybrid models such as TDS+BiLSTM and ConvLSTMConv/ConvGRUConv, and attention-heavy alternatives including BiLSTM+Transformer and a Conformer. Both plain and CNN-front-end vanilla BiRNN controls were retained as negative controls to show what failed in this regime. Architecture selection was followed by recurrent-focused ablations over four factors: temporal resolution, context design, recurrent architecture, and data availability.

For temporal resolution, we swept spectrogram hop length from 8 to 64 and confirmed the strongest region with longer training. For context design, we ran a broader sweep over BiLSTM windows of 8000 and 16000 with paddings [1800,200] and [900,100], and BiGRU windows of 4000, 12000, and 20000 with strides 2000, 6000, and 12000 and paddings [900,100] or [900,200]. This was intended to separate the effects of longer context, contextual padding, and window overlap. For recurrent architecture, we compared LSTM versus GRU, different hidden sizes and depths, and the addition of convolutional blocks around the recurrent core. For data availability, we reduced the number of channels and training sessions while keeping the backbone fixed to test whether remaining limitations were architectural or informational.

2.3 Decoder Design

Decoder-side improvements used the repository’s beam-search decoder with a character-level 6-gram language model (`wikitext-103-6gram-charlm.bin`). We swept beam size over {10, 25, 50, 75, 100} with LM weight 2.0 and insertion bonus 2.0. This let us isolate decoder gain on a fixed encoder before introducing the Raw-CNN+BiGRU family.

2.4 Final Learned-Downsampling Raw-CNN+BiGRU Design

The learned-downsampling Raw-CNN+BiGRU replaced the handcrafted spectrogram front end with three 1D convolution layers followed by a two-layer bidirectional GRU with hidden size 384. All Raw-CNN candidates used kernels [9, 9, 9], strides [3, 4, 4], paddings [4, 4, 4], and batch normalization, so the total stride remained $3 \times 4 \times 4 = 48$, deliberately mirroring the effective temporal downsampling that emerged as best in the spectrogram hop-length sweep. We used an increasing channel schedule so that representational capacity grows as the temporal dimension is progressively downsampled, following a standard CNN design pattern. To keep this design choice grounded empirically, we evaluated two width schedules, [64, 128, 192] and [96, 192, 256], under the same kernel and stride configuration, and then carried the stronger setting forward for the rest of the Raw-CNN+BiGRU study.

The final confirmed Raw-CNN+BiGRU therefore used channels [96, 192, 256], `window_length=20000`, `stride=12000`, and `padding=[900,200]`, corresponding to 40% window overlap. We did not resweep raw-

model context settings separately; instead, the raw-input family inherited this strongest long-context setting from the earlier spectrogram recurrent ablation. After this width schedule had already produced the strongest screening result in this family, we did not pursue a broader Raw-CNN width sweep; instead, we used the remaining budget for the GRU dropout sweep $\{0.0, 0.5\}$, longer 100- and 120-epoch confirmation runs, and post-hoc beam decoding.

3 Results

All CER values below are percentages. We explicitly label 40-epoch runs as screening runs whenever they are compared against longer confirmation runs.

3.1 Architecture Selection

The first question was whether the course TDS CNN baseline should remain the main architecture family. Table 1 shows that the answer is no. Recurrent encoders consistently outperform the baseline, while attention-heavy alternatives and ungated recurrent controls either converge more slowly or generalize worse.

Table 1: Representative architecture comparison.

Family	Representative configuration	Horizon		Params	Val CER	Test CER
TDS CNN baseline	TDS CNN, kernel 32	150-epoch	confir- mation	5.3M	18.94	22.17
Vanilla BiRNN control	BiRNN, $h=512$, $l=5$, tanh	150-epoch	confir- mation	8.1M	45.70	37.00
BiLSTM	$h=384$, $l=2$, $hop=16$, $win=8000$	150-epoch	confir- mation	8.2M	14.55	15.76
TDS+BiLSTM	warm-started hybrid	150-epoch	confir- mation	13.0M	14.58	15.47
BiLSTM+Transformer	BiLSTM + 2 Transformer layers	150-epoch	confir- mation	22.3M	14.67	17.25
Conformer	2-layer Conformer, kernel 31	40-epoch	screen- ing	15.1M	30.77	26.78
BiGRU	$h=384$, $l=2$, $hop=48$, $win=16000$, $pad=[900,100]$	150-epoch	confir- mation	6.4M	13.49	14.11
CNN+vanilla BiRNN control	TDS CNN + vanilla BiRNN + 2 FC layers	150-epoch	confir- mation	9.5M	31.52	25.20

Two patterns are immediate. First, gating is the decisive architectural change, not recurrence alone: even after 150 epochs, the vanilla BiRNN control remains at 45.70 / 37.00, and the CNN+vanilla BiRNN control reaches only 31.52 / 25.20. By contrast, the plain BiLSTM already closes most of the gap between the baseline and the later best systems. Second, simply adding more complexity on top of a gated recurrent model does not help in proportion to parameter count. The TDS+BiLSTM hybrid barely improves on BiLSTM, while the BiLSTM+Transformer stack slightly improves validation but worsens test CER. This motivated the rest of the project to focus on recurrent models and the design choices within that family.

3.2 Ablation Studies

Once the search narrowed to recurrent encoders, the most important question became which low-level design decisions actually mattered. Rather than listing every intermediate run first, we focus on the ablation trends that most directly shaped the final system design.

The hop-length sweep shows that lower temporal resolution is consistently better once the encoder is recurrent. The baseline $hop=16$ setting performs worse than $hop=32$, $hop=48$, and $hop=64$, and the longer confirmation run at $hop=48$ shows that this is not merely an early-training artifact. Instead, the model

benefits from shorter sequences and reduced frame-level redundancy. The later Raw-CNN+BiGRU design was built directly on this observation.

Context design matters just as much as hop length, and the underlying sweep was broader than the abbreviated table alone might suggest. In the BiLSTM sweep, larger windows generally helped and the smaller padding choice [900,100] generalized better than [1800,200]. In the later BiGRU sweep, the more subtle issue was overlap. The clearest failure case was `window_length=12000, stride=2000, padding=[900,200]`, which implies about 83% overlap and reached 14.02 validation CER but 49.84 test CER. By contrast, 4000/2000 (50% overlap) gave 16.64 / 15.34, 12000/6000 (50% overlap) gave 15.47 / 16.99, and 20000/12000 (40% overlap) gave the best GRU screening result at 13.83 / 15.09. The safest conclusion is therefore not that every overlap above 50% fails, but that long windows with very dense overlap can make validation overly optimistic. Since train and validation are windowed while test is evaluated on full sessions without windowing or contextual padding, reducing redundancy appears to reduce this train/validation/test mismatch.

Within the recurrent family, compact BiGRU models generalize slightly better than larger or more complicated alternatives. The best BiGRU beats the best BiLSTM on test CER while using fewer parameters, and adding convolutional blocks around the GRU core does not improve test performance. The data ablations point in the same direction: reducing channels or training sessions sharply degrades CER, which indicates that the regime is information-limited rather than capacity-limited. In other words, the best models are the ones that use limited signal and limited data efficiently, not the ones that simply maximize parameter count.

Representative ablation runs behind these conclusions are summarized in Table 2.

Table 2: Key ablation results across temporal resolution, context design, recurrent architecture, and data availability.

Study		Representative configuration	Horizon	Val CER	Test CER
Hop baseline		BiLSTM <code>h=384, l=2, hop=16, win=8000</code>	40-epoch screening	19.87	20.08
Lower frame rate		BiLSTM <code>h=384, l=2, hop=48, win=8000</code>	40-epoch screening	17.01	17.59
Confirmed	low frame rate	BiLSTM <code>h=384, l=2, hop=48, win=8000</code>	150-epoch confirmation	13.98	14.52
Best BiLSTM context	con-	BiLSTM <code>h=384, l=2, hop=48, win=16000, pad=[900,100]</code>	150-epoch confirmation	13.65	14.93
Overlapped context	GRU	BiGRU <code>h=384, l=2, hop=48, win=12000, pad=[900,200], stride=2000</code>	40-epoch screening	14.02	49.84
Best spectrogram GRU context		BiGRU <code>h=384, l=2, hop=48, win=20000, pad=[900,200], stride=12000</code>	40-epoch screening	13.83	15.09
Best spectrogram encoder		BiGRU <code>h=384, l=2, hop=48, win=16000, pad=[900,100]</code>	150-epoch confirmation	13.49	14.11
Conv around GRU core		ConvGRUConv, matched low-rate context	150-epoch confirmation	13.98	15.06
Channel reduction		BiLSTM <code>h=384, l=2, 8 channels/band</code>	40-epoch screening	25.88	26.35
Training-data reduction	re-	BiLSTM <code>h=384, l=2, 8 train sessions</code>	40-epoch screening	36.97	33.46

3.3 Decoder Study

We next separated decoder gain from encoder gain. Throughout this section, greedy CTC results are used to compare trained encoders, while beam-search results are post-hoc offline decoding runs on fixed checkpoints rather than separate training runs. Applying beam search to the best spectrogram BiGRU checkpoint produces one of the largest single improvements in the entire project. Across beam sizes {10, 25, 50, 75, 100}, the validation/test CERs are 9.30/8.90, 8.75/8.88, 8.46/8.69, 8.40/8.84, and 8.40/8.77, respectively. Larger beams slightly improve validation but do not improve test in the same way, so `beam=50` is the best

Table 3: Raw-CNN+BiGRU family summary: greedy tuning results and post-hoc beam decoding.

Raw-CNN+BiGRU variant	Key settings	Horizon	Val CER	Test CER
Narrow width base-line	<code>channels=[64,128,192]</code> , <code>gru_dropout=0.0</code>	40-epoch screening	11.77	13.40
Wider front end	<code>channels=[96,192,256]</code> , <code>gru_dropout=0.0</code>	40-epoch screening	10.64	12.60
Longer no-dropout confirmation	<code>channels=[96,192,256]</code> , <code>gru_dropout=0.0</code>	100-epoch confirmation	10.013	12.622
Final best greedy encoder	<code>channels=[96,192,256]</code> , <code>gru_dropout=0.5</code>	120-epoch confirmation	9.933	11.476
Best decoded 100-epoch checkpoint	<code>channels=[96,192,256]</code> , <code>gru_dropout=0.0</code> + beam search	Offline decoding	6.627	7.240
Best decoded 120-epoch checkpoint	<code>channels=[96,192,256]</code> , <code>gru_dropout=0.5</code> + beam search	Offline decoding	6.720	6.938

speed-accuracy trade-off. Relative to greedy decoding on the same encoder, beam search at 50 reduces validation CER by 5.03 points and test CER by 5.42 points.

After selecting the strongest Raw-CNN+BiGRU checkpoints under greedy decoding, we applied the same decoder to those saved checkpoints as a final system-level evaluation. The no-dropout Raw-CNN+BiGRU decodes to 6.627 / 7.240, while the `gru_dropout=0.5` Raw-CNN+BiGRU decodes to 6.720 / 6.938. Since the final submission is determined by the lowest test CER, the submitted system should be the `gru_dropout=0.5` Raw-CNN+BiGRU with beam decoding, which achieves 6.938 on test.

3.4 Final Learned-Downsampling Raw-CNN+BiGRU

The final step was to turn the temporal-resolution insight into a learned front end. To keep this subsection focused on encoder design rather than decoding, all results below are greedy CTC results from the trained Raw-CNN+BiGRU family itself.

We first ran a narrow Raw-CNN width comparison under matched kernel and stride settings. A smaller `[64,128,192]` Raw-CNN+BiGRU reached 11.77 / 13.40 after 40 screening epochs, while the wider `[96,192,256]` variant improved that result to 10.64 / 12.60. This supports the design choice of increasing channels as temporal resolution is reduced: the wider schedule gave the network more representational capacity while keeping the same overall downsampling behavior. We therefore carried `[96,192,256]` forward for the remaining encoder-side tuning.

Table 3 should be read in two blocks. The first four rows are greedy CTC results and show the encoder-side tuning path: increasing the CNN width improves the 40-epoch screening result at fixed stride, and the later confirmation runs continue that trend. Compared with the earlier 100-epoch no-dropout confirmation, the later 120-epoch `gru_dropout=0.5` run reaches the best greedy result in this family at 9.933 / 11.476. The last two rows are post-hoc beam-decoded results on those same saved checkpoints. They show that the 100-epoch no-dropout checkpoint gives the best decoded validation CER, while the 120-epoch `gru_dropout=0.5` checkpoint gives the best decoded test CER at 6.938. Under greedy decoding alone, the final Raw-CNN+BiGRU already improves on the best spectrogram encoder (13.49 / 14.11), and beam decoding then provides an additional system-level gain on top of that encoder improvement.

4 Discussion

Recurrent inductive bias. The strongest consistent result in this project is that gated recurrent models are the right architectural bias for this single-user problem. Compared with the TDS CNN baseline, recurrent encoders can model temporal dependencies directly across the full sequence instead of relying on a mostly local receptive field. At the same time, the attention-heavy alternatives we tested appear too data-hungry for this regime. The Conformer underperforms badly even in screening, and the BiLSTM+Transformer stack adds considerable complexity without improving test CER. Model size alone also does not explain the

ranking: the 6.4M BiGRU outperforms substantially larger 13.0M and 22.3M hybrids, which is consistent with a data-limited single-user regime where extra capacity is not the main bottleneck. This combination suggests that the bottleneck is not missing global-attention capacity; it is how efficiently the model can use limited signal and limited data.

Temporal resolution and context. The hop-length and context ablations provide a second important insight: more temporal detail is not always better. Lower effective frame rates outperform the inherited high-rate baseline, and longer windows help when overlap is controlled. An additional nuance is the evaluation protocol: train and validation use the configured windowing scheme, but test is evaluated on full sessions without windowing or contextual padding. This likely explains why heavily overlapped long-window settings can look reasonable on validation yet collapse on test. The strongest context setting was not chosen because it kept overlap above 50%; for `window_length=20000` and `stride=12000`, the overlap is actually 40%. Its advantage is that it combines long context with lower redundancy and a smaller train/validation/test mismatch. This interpretation also explains why the `hop=48` result was a productive bridge to the Raw-CNN+BiGRU family: once the project had learned that a coarser temporal grid was beneficial, it became natural to ask whether the model should learn that compression itself rather than inheriting it from a fixed transform.

Learned raw downsampling. That is exactly what the final Raw-CNN+BiGRU demonstrates. The learned-downsampling front end does not win simply because it is “different”; it wins because it turns the best earlier ablation result into a learned module. By matching the successful 48x temporal reduction with convolutional strides, the model keeps the favorable sequence length while allowing the front end to optimize task-specific features directly from raw EMG. The increasing channel schedule fits the same logic: as the temporal axis is compressed, the model can expand feature capacity in the channel dimension instead. The result is a cleaner encoder gain than we obtained from larger LSTMs, extra convolutional blocks around the spectrogram encoder, or attention-heavy models. The improvement therefore supports a specific hypothesis: in this regime, the key advantage is not extra depth or extra modules in isolation, but learned temporal compression that respects the timescale of the task.

Decoder gains and limitations. Beam search adds a complementary lesson. Its improvement on the spectrogram BiGRU is large, and the decoded Raw-CNN+BiGRU results show that decoder-side language information remains useful even after strong encoder improvements. At the same time, beam search does not eliminate the need for a better encoder; the Raw-CNN+BiGRU still improves the greedy starting point substantially. There are also clear limitations. All results are from a single-user setting, so we cannot claim the same ranking will hold across users. The best decoded validation and test results come from different Raw-CNN+BiGRU checkpoints, which is a reminder that this regime is small enough for checkpoint selection noise to matter. The Raw-CNN width search was also intentionally narrow: we compared [64, 128, 192] against [96, 192, 256] and then carried the stronger variant forward rather than performing an exhaustive width sweep. Finally, our best results rely on offline LM-based decoding, so the reported numbers do not by themselves establish a real-time deployment point. Even with those caveats, the central story remains stable: recurrent models beat the baseline, coarse temporal compression matters, and learned raw downsampling is the most effective encoder-side improvement we found.

References

- [1] Alex Graves, Santiago Fernandez, Faustino Gomez, and Jurgen Schmidhuber. Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. *Proceedings of the 23rd International Conference on Machine Learning*, 2006.
- [2] Viswanath Sivakumar, Jeffrey Seely, Alan Du, Sean R. Bittner, Adam Berenzweig, Anuoluwapo Bolarinwa, Alexandre Gramfort, and Michael I. Mandel. emg2qwerty: A large dataset with baselines for touch typing using surface electromyography. *arXiv preprint arXiv:2410.20081*, 2024.
- [3] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.

- [4] Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, William Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, and Ruoming Pang. Conformer: Convolution-augmented transformer for speech recognition. *Proceedings of Interspeech*, 2020.